Chapter 12: Polymorphism, Virtual Functions and Abstract Classes

C++ PROGRAMMING:

Objectives

- Learn about Polymorphism
- Learn about virtual functions
- Learn about virtual destructor
- Rules of virtual functions
- Become aware of abstract classes

Polymorphism and Virtual Function

- Polymorphism principle refers to that objects (derived objects) belonging to different classes are able to respond to the same message but in different forms.
- We use a pointer or reference to base class to refer to all the derived objects.
- The goal of the polymorphism is ***generalization***
- Polymorphism means “many forms”

Polymorphism and Virtual Function [con't]

- Polymorphism Requirements:
  1. There must be an inheritance hierarchy
  2. The base class in the hierarchy must have a virtual method with the same signature to its corresponding method in the derived class
  3. There must be either a pointer or a reference to the base class object. This pointer of reference is used t invoke a virtual function.

Polymorphism and Virtual Function [con't]

- A virtual function is like other functions but it declared with reserved word **virtual**.
- When two methods with same function signature in both the base and derived classes the method in the base class is declared as ***virtual***
- Compiler determines which virtual functions to use at run time based on the type of object pointed to by base pointer rather than the type of the pointer.
- By making the base pointer points to different derived objects we can execute different versions of the virtual function

Polymorphism and Virtual Function [con't]

- Compile-time binding: the necessary code to call specific function is generated by compiler
  - Also known as static binding or early binding

- Virtual function: binding occurs at program execution time, not at compile time

Polymorphism and Virtual Function [con't]

```
#include<iostream>
using namespace std;

class X
{
public:
    X() { a=1; }
    virtual void print ()  {cout<<"Inside Class X \n";}
private:
    int a;
};

class Y: public X
{
public:
    Y() { b=2; }
    void print ()  {cout<<"Inside Class Y \n";}
private:
    int b;
};

void test (X& xx)
{
    xx.print();
}
```

```
int main()
{
    X *px1, *px2;

    px1 = new X();
    px1->print();

    px2 = new Y();
    px2->print();
}
```

OUTPUT

```
Inside Class X
Inside Class Y
```

```
int main()
{
    X x1; Y y1;

    test(x1);
    test(y1);
}
```

OUTPUT

```
Inside Class X
Inside Class Y
```

Polymorphism and Virtual Function [con't]

```
#include<iostream>
using namespace std;
class Point
{
public:
    Point(int xVal=0, int yVal=0)
    {
        x=xVal; y=yVal;
    }

    virtual double area()    { return 0;  }

    virtual double volume()  { return 0;  }

    virtual void printShapeName()
    {
        cout<<"Point"<<endl;
    }

    virtual void print()
    {
        cout<<'('<<x<<','<<y<<')'<<endl;
    }
protected:
    int x;
    int y;
};
```

```
class Circle: public Point
{
public:

    Circle (int xVal=0, int yVal=0, int rr=0): Point (xVal,yVal)
    {
        r=rr;
    }

    double area()    { return r*r*3.14; }

    double volume()  { return 0;   }

    void printShapeName()
    {
        cout<<"Circle"<<endl;
    }

    void print()
    {
        Point::print();
        cout<<"Radius: "<<r<<endl;
    }

private:
    int r;
};
```

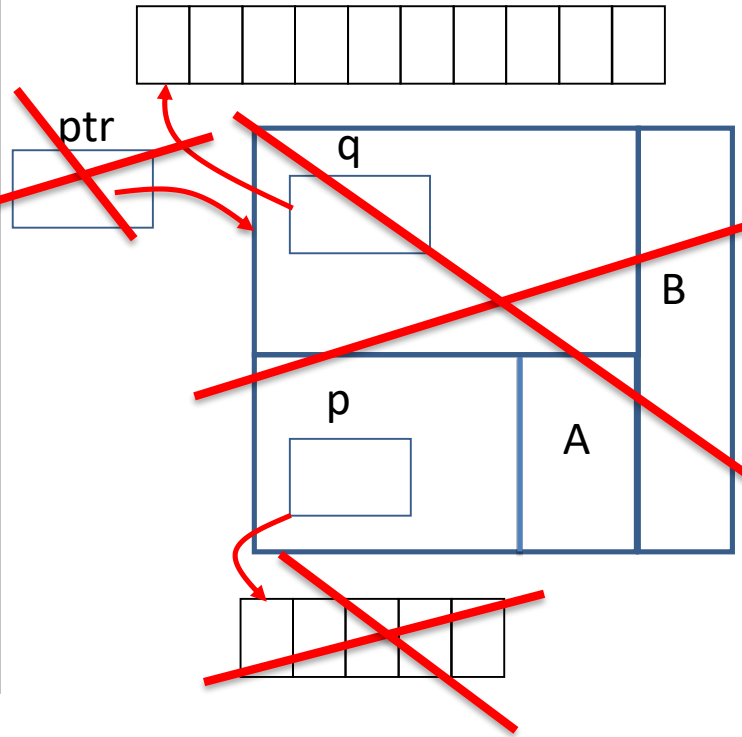
```
int main()
{
    Point* arr[2];
    arr[0]= new Point(3,4);
    arr[1]= new Circle(3,3,5);

    for (int i=0; i<2; i++)
    {
        arr[i]->printShapeName
        arr[i]->print();
        cout<<"Area: "<<arr[i]
        cout<<"Volume: "<<arr[
        cout<<"--------------
    }

    return 0;
}
```

```
Point
(3,4)
Area: 0
Volume: 0
----------
Circle
(3,3)
Radius: 5
Area: 78.5
Volume: 0
----------
```

Virtual Destructors

```
#include<iostream>
using namespace std;

class A
{
public:
    A()
    {
        cout<<"A's Constructor Firing"<<endl;
        p = new char[5];
    }
    ~A()
    {
        cout<<"A's Destructor Firing"<<endl;
        delete []p;
    }

private:
    char* p;
};
```

```
class B: public A
{

public:
        B()
        {
            cout<<"B's Constructor Firing"<<endl;
            q = new char[10];
        }
        ~B()
        {
            cout<<"B's Destructor Firing"<<endl;
            delete []q;
        }

private:
    char* q;
};
```




```
int main()
{
      A* ptr;
      ptr = new B();
      delete ptr;

}
```

OUTPUT

```
A's Constructor Firing
B's Constructor Firing
A's Destructor Firing
```

B's Destructor was not called

Virtual Destructors

- The call ***new*** operator cause the constructor *A()* and *B()* to fire
- When we invoke ***delete*** operator on *ptr* only *~A()* fire despite the fact that *ptr* points to *B* object because the destructor are not virtual
- The compiler uses ptr's data type to (i.e., A*) to determine which destruct to call thus only *~A()* is called.
- The problem with previous program can be fixed by making the base destructor virtual

Virtual Destructors [con't]

```
#include<iostream>
using namespace std;

class A
{
public:
    A()
    {
        cout<<"A's Constructor Firing"<<endl;
        p = new char[5];
    }
    virtual ~A()
    {
        cout<<"A's Destructor Firing"<<endl;
        delete []p;
    }

private:
    char* p;
};
```

```
class B: public A
{

public:
        B()
        {
            cout<<"B's Constructor Firing"<<endl;
            q = new char[10];
        }
        ~B()
        {
            cout<<"B's Destructor Firing"<<endl;
            delete []q;
        }

private:
    char* q;
};
```

```
int main()
{
      A* ptr;
      ptr = new B();
      delete ptr;

}
```

OUTPUT

```
A's Constructor Firing
B's Constructor Firing
B's Destructor Firing
A's Destructor Firing
```

Virtual Destructors [con't]

- Virtual destructor of a base class automatically makes the destructor of a derived class virtual
  - After executing the destructor of the derived class, the destructor of the base class executes
- If a base class contains virtual functions, make the destructor of the base class virtual

Rules of Virtual Functions

1. The virtual functions must be members of some class
2. They can not be static
3. The prototypes of the base version of a virtual function an all the derived class versions must be identical
4. We can not have virtual constructors but we can have virtual destructors

Abstract Classes and Pure Virtual Functions

- New classes can be derived through inheritance without designing them from scratch
  - Derived classes inherit existing members of base class
  - Can add their own members
  - Can redefine or override public and protected member functions
- Base class can contain functions that you would want each derived class to implement
  - However, base class may contain functions that may not have meaningful definitions in the base class

Abstract Classes and Pure Virtual Functions (cont'd.)

- Pure virtual functions:
  - Do not have definitions (bodies have no code)
- Example: `virtual void draw() = 0;`
- Abstract class: a class with one or more virtual functions
  - Can contain instance variables, constructors, and functions that are not pure virtual
  - Class must provide the definitions of constructor/functions that are not pure virtual

Abstract Classes and Pure Virtual Functions

```
#include<iostream>
using namespace std;
class Point
{
public:
    Point(int xVal=0, int yVal=0)
    {
        x=xVal; y=yVal;
    }

    virtual double area()=0;
    virtual double volume()=0;

    virtual void printShapeName()
    {
        cout<<"Point"<<endl;
    }

    virtual void print()
    {
        cout<<'('<<x<<','<<y<<')'<<endl;
    }
protected:
    int x;
    int y;
};
```

```
class Circle: public Point
{
public:

    Circle (int xVal=0, int yVal=0, int rr=0): Point (xVal,yVal)
    {
        r=rr;
    }

    double area()    { return r*r*3.14; }
    double volume()  { return 0;   }

    void printShapeName()
     {
        cout<<"Circle"<<endl;
     }

     void print()
     {
        Point::print();
        cout<<"Radius: "<<r<<endl;
     }

private:
    int r;
};
```

```
int main()
{
    Circle c(2,4,6);
    c.printShapeName();
    cout<<"Area: "<<c.area()<<endl;
    cout<<"Volume: "<<c.volume()<<endl;
    c.print();

    return 0;
}
```

OUTPUT

```
Circle
Area: 113.04
Volume: 0
(2,4)
Radius: 6
```

Abstract Classes and Pure Virtual Functions

you cannot create objects from the abstract class

Summary (cont'd.)

- Dynamic variable: created during execution
  - Created using `new`, deallocated using `delete`
- Shallow copy: two or more pointers of the same type point to the same memory
- Deep copy: two or more pointers of the same type have their own copies of the data
- Binding of virtual functions occurs at execution time (dynamic or run-time binding)

The End